

**LAPORAN PRAKTIKUM MEMBUAT DAN MENGELOLA LXC CONTAINER
PADA PROXMOX VIRTUAL ENVIRONMENT (PROXMOX VE)**



Disusun oleh:

Nama : Adrian Saputra

Kelas : TI 4A

NIM : 2402001

PROGRAM STUDI TEKNIK INFORMATIKA

POLITEKNIK PURBAYA

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum "Membuat dan Mengelola LXC Container pada Proxmox Virtual Environment" ini dapat diselesaikan. Laporan ini disusun sebagai pemenuhan tugas praktikum yang membagi materi virtualisasi Proxmox VE menjadi empat bagian terpisah, yaitu instalasi Proxmox, pengelolaan mesin virtual (VM), pengelolaan LXC Container, dan konfigurasi jaringan.

Perlu disampaikan secara transparan bahwa pada saat penyusunan laporan ini, penulis belum sempat melaksanakan praktik langsung pembuatan dan pengelolaan LXC Container di lingkungan Proxmox VE akibat keterbatasan waktu. Oleh karena itu, laporan ini disusun berdasarkan kajian teori yang mendalam serta panduan langkah kerja standar yang seharusnya ditempuh, sebagai kerangka acuan pelaksanaan praktik. Penulis menyadari bahwa idealnya setiap tahapan disertai bukti tangkapan layar (screenshot) hasil praktik langsung, dan hal ini menjadi catatan tindak lanjut yang dijelaskan lebih rinci pada bagian Batasan Laporan dan Bab VI Penutup.

Penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah yang telah memberikan bimbingan, serta semua pihak yang telah membantu proses penyusunan laporan ini. Penulis menyadari laporan ini masih jauh dari sempurna, sehingga kritik dan saran yang membangun sangat diharapkan demi perbaikan di kemudian hari.

Semoga laporan ini dapat bermanfaat bagi pembaca dan menjadi acuan pelaksanaan praktik pengelolaan container di masa mendatang.

Tegal, Juli 2026
Penulis

DAFTAR ISI

KATA PENGANTAR.....	1
DAFTAR ISI	2
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Rumusan Masalah.....	2
1.3 Tujuan.....	2
1.4 Manfaat.....	3
1.5 Batasan Laporan	3
BAB II DASAR TEORI	4
2.1 Virtualisasi dan Kontainerisasi.....	4
2.2 Konsep LXC (Linux Container)	4
2.3 Perbedaan LXC dengan Virtual Machine.....	5
2.4 Arsitektur LXC pada Proxmox VE	5
2.5 Template Container	6
2.6 Manajemen Sumber Daya Container.....	7
2.7 Snapshot dan Cloning Container	7
2.8 Keamanan Container	7
BAB III ALAT DAN BAHAN	8
Rencana Alokasi Sumber Daya Container	8
BAB IV LANGKAH-LANGKAH PRAKTIKUM	1
4.1 Persiapan Template Container.....	1
4.2 Pembuatan Container LXC Baru.....	2
4.3 Menjalankan dan Mengakses Container.....	3
4.4 Pengelolaan Siklus Hidup Container.....	3
4.5 Instalasi Paket di Dalam Container	4
4.6 Pengaturan Ulang Sumber Daya (Resource)	4
4.7 Snapshot Container.....	5
4.8 Cloning Container.....	5
BAB V HASIL DAN PEMBAHASAN	6
5.1 Ringkasan Status Pelaksanaan Tahapan	6
5.2 Antisipasi Kendala Berdasarkan Pengalaman Praktik Sebelumnya.....	6
5.3 Analisis Perbandingan Efisiensi terhadap Virtual Machine	7

BAB VI PENUTUP.....	8
6.1 Kesimpulan.....	8
6.2 Saran	9
DAFTAR PUSTAKA.....	10
LAMPIRAN	11
Lampiran A — Rencana Perintah Shell untuk Verifikasi Container.....	11
Lampiran B — Perbandingan Ringkas dengan Tiga Bagian Laporan Lain	11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi virtualisasi telah mengubah cara pengelolaan sumber daya komputasi secara signifikan. Salah satu bentuk virtualisasi yang semakin populer adalah kontainerisasi (containerization), yaitu teknik menjalankan aplikasi atau sistem operasi dalam lingkungan terisolasi yang berbagi kernel dengan sistem operasi host, tanpa memerlukan hypervisor penuh seperti pada mesin virtual (Virtual Machine/VM) konvensional.

Proxmox Virtual Environment (Proxmox VE) sebagai salah satu platform virtualisasi open-source berbasis Debian menyediakan dua jenis mekanisme virtualisasi sekaligus dalam satu platform, yaitu KVM (Kernel-based Virtual Machine) untuk VM penuh dan LXC (Linux Container) untuk kontainerisasi ringan. Kehadiran LXC pada Proxmox VE memberikan alternatif yang jauh lebih efisien dari sisi penggunaan sumber daya (CPU, RAM, dan disk) dibandingkan VM konvensional, terutama untuk menjalankan layanan yang tidak memerlukan kernel terpisah.

Dalam rangkaian praktikum mata kuliah ini, materi Proxmox VE dibagi menjadi empat bagian besar, yaitu instalasi Proxmox, pengelolaan Virtual Machine, pengelolaan LXC Container, dan konfigurasi jaringan. Laporan ini secara khusus membahas bagian ketiga, yaitu proses membuat dan mengelola LXC Container, mencakup pembuatan container dari template, pengelolaan siklus hidup (start, stop, reboot), instalasi paket, pengaturan ulang sumber daya, snapshot, hingga cloning container.

Sebagai catatan kejujuran akademik, penyusunan laporan ini dilakukan berdasarkan kajian teori dan prosedur standar operasional LXC pada Proxmox VE, sebagai kerangka acuan pelaksanaan praktik. Keterbatasan waktu menyebabkan praktik langsung beserta dokumentasi tangkapan layar belum sempat dilaksanakan secara penuh pada saat laporan ini disusun. Hal ini dijelaskan lebih lanjut pada bagian 1.5 Batasan Laporan.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam praktikum ini adalah sebagai berikut:

1. Bagaimana konsep dan arsitektur LXC Container bekerja di atas platform Proxmox VE?
2. Bagaimana prosedur pembuatan container LXC baru menggunakan template sistem operasi yang tersedia?
3. Bagaimana cara mengelola siklus hidup container (start, stop, reboot) serta menginstal paket perangkat lunak di dalamnya?
4. Bagaimana cara mengubah alokasi sumber daya (CPU dan memori) suatu container yang sudah berjalan?
5. Bagaimana mekanisme snapshot dan cloning container bekerja, serta apa manfaatnya dalam pengelolaan infrastruktur?

1.3 Tujuan

Tujuan dari penyusunan laporan praktikum ini adalah:

1. Memahami konsep dasar kontainerisasi dan perbedaannya dengan virtualisasi berbasis hypervisor.
2. Mengetahui dan mampu menerapkan prosedur pembuatan LXC Container pada Proxmox VE menggunakan template resmi.
3. Mampu mengelola siklus hidup container serta melakukan instalasi paket perangkat lunak di dalamnya.
4. Mampu mengubah dan menyesuaikan alokasi sumber daya container sesuai kebutuhan beban kerja.
5. Memahami dan mampu menerapkan mekanisme snapshot serta cloning container untuk keperluan backup dan replikasi lingkungan.

1.4 Manfaat

Laporan ini diharapkan memberikan manfaat sebagai berikut:

1. Bagi penulis, sebagai sarana pendalaman pemahaman teori dan praktik pengelolaan container pada infrastruktur virtualisasi modern.
2. Bagi pembaca, sebagai referensi dan panduan langkah kerja standar dalam membuat serta mengelola LXC Container di Proxmox VE.
3. Bagi pengembangan keilmuan, sebagai bahan pembanding antara pendekatan kontainerisasi dengan virtualisasi penuh dalam konteks efisiensi sumber daya.

1.5 Batasan Laporan

Untuk menjaga transparansi dan integritas akademik, penulis menyampaikan batasan-batasan berikut terkait laporan ini:

1. Pada saat penyusunan laporan, praktik langsung pembuatan dan pengelolaan LXC Container di lingkungan Proxmox VE belum sempat dilaksanakan sepenuhnya akibat keterbatasan waktu, sehingga laporan ini belum dilengkapi dengan tangkapan layar (screenshot) hasil praktik nyata.
2. Seluruh langkah kerja pada Bab IV disusun berdasarkan prosedur operasional standar (SOP) resmi Proxmox VE dan referensi teknis yang berlaku umum, sehingga dapat dijadikan acuan pelaksanaan praktik susulan.
3. Bab V (Hasil dan Pembahasan) disusun dalam bentuk analisis prediktif berdasarkan teori dan pengalaman troubleshooting pada praktik VM sebelumnya (lihat laporan poin 2), bukan hasil eksekusi aktual pada container.
4. Penulis merekomendasikan agar laporan ini diperbarui dengan bukti dokumentasi (screenshot) dan data hasil aktual segera setelah praktik LXC Container dilaksanakan, sebagaimana dijelaskan pada Bab VI Penutup.

BAB II

DASAR TEORI

2.1 Virtualisasi dan Kontainerisasi

Virtualisasi adalah teknik menciptakan representasi virtual dari sumber daya komputasi fisik (server, storage, jaringan) sehingga satu perangkat keras dapat menjalankan beberapa sistem operasi atau lingkungan kerja secara bersamaan dan terisolasi. Terdapat dua pendekatan utama virtualisasi yang relevan dalam konteks Proxmox VE, yaitu virtualisasi berbasis hypervisor (seperti KVM) dan kontainerisasi berbasis kernel yang dibagi bersama (seperti LXC).

Kontainerisasi adalah bentuk virtualisasi di level sistem operasi (OS-level virtualization) yang memungkinkan beberapa lingkungan terisolasi (container) berjalan di atas satu kernel sistem operasi host yang sama. Setiap container memiliki ruang nama (namespace) proses, jaringan, dan filesystem-nya sendiri, namun tidak menjalankan kernel terpisah seperti pada VM. Pendekatan ini membuat container jauh lebih ringan, cepat dibuat, dan hemat sumber daya dibandingkan VM.

2.2 Konsep LXC (Linux Container)

LXC (Linux Containers) adalah teknologi virtualisasi tingkat sistem operasi yang memanfaatkan fitur kernel Linux seperti cgroups (control groups) untuk pembatasan dan prioritas sumber daya, serta namespaces (pid, net, mnt, uts, ipc, user) untuk isolasi proses, jaringan, dan filesystem. Proxmox VE mengintegrasikan LXC sebagai salah satu mesin virtualisasi utamanya, sehingga administrator dapat membuat, menjalankan, dan mengelola container melalui antarmuka web yang sama dengan pengelolaan VM berbasis KVM.

Setiap container LXC dibuat dari sebuah template, yaitu arsip sistem berkas (root filesystem) suatu distribusi Linux yang telah disiapkan dan dikemas sebelumnya. Proxmox VE menyediakan repositori template resmi untuk berbagai distribusi seperti Debian, Ubuntu, CentOS, Alpine, dan lainnya, yang dapat diunduh langsung melalui antarmuka web tanpa proses instalasi manual seperti pada VM.

2.3 Perbedaan LXC dengan Virtual Machine

Perbedaan mendasar antara LXC dan VM berbasis KVM terletak pada tingkat isolasi dan penggunaan sumber daya. VM menjalankan kernel sistem operasi tamu (guest) sendiri secara penuh di atas hypervisor, sehingga isolasinya lebih kuat namun memerlukan overhead sumber daya yang lebih besar. Sebaliknya, LXC berbagi kernel dengan host, sehingga proses booting jauh lebih cepat (hitungan detik, bukan menit) dan konsumsi RAM/CPU jauh lebih efisien.

Aspek	LXC Container	Virtual Machine (KVM)
Kernel	Berbagi kernel dengan host	Kernel tamu terpisah sepenuhnya
Waktu Booting	Sangat cepat (hitungan detik)	Lebih lambat (puluhan detik–menit)
Penggunaan RAM/CPU	Sangat efisien, overhead rendah	Lebih besar karena emulasi hardware penuh
Isolasi	Isolasi tingkat proses (namespace)	Isolasi penuh tingkat hardware virtual
Kompatibilitas OS Tamu	Terbatas pada OS berbasis Linux (kernel sama)	Fleksibel, bisa OS apa saja (Windows, BSD, dll)
Kasus Penggunaan Ideal	Layanan ringan: web server, database, aplikasi microservice	Aplikasi yang butuh kernel/OS berbeda atau isolasi ketat

Berdasarkan perbandingan pada Tabel 2.1, dapat disimpulkan bahwa LXC lebih cocok digunakan untuk skenario yang mengutamakan efisiensi dan kecepatan, seperti menjalankan banyak layanan ringan sekaligus dalam satu server fisik, sedangkan VM lebih tepat digunakan ketika dibutuhkan isolasi penuh atau sistem operasi yang berbeda dari host.

2.4 Arsitektur LXC pada Proxmox VE

Pada Proxmox VE, setiap container LXC diberi identitas berupa CT ID (Container ID) yang unik dalam satu cluster/node, serupa dengan VM ID pada VM berbasis KVM. Container dapat dikonfigurasi dengan alokasi CPU (jumlah core), memori (RAM dan swap), penyimpanan (rootfs pada storage seperti local-lvm), serta antarmuka jaringan yang terhubung ke bridge virtual (misalnya vmbr0), sehingga container dapat berkomunikasi dengan jaringan fisik maupun VM lain dalam satu host.

Proxmox VE mendukung dua mode isolasi container, yaitu privileged dan unprivileged. Container privileged memiliki hak akses root yang secara langsung dipetakan ke root pada

sistem host, sehingga lebih berisiko dari sisi keamanan namun lebih kompatibel untuk kebutuhan tertentu. Container unprivileged memetakan UID/GID di dalam container ke rentang UID/GID non-root pada host, sehingga jauh lebih aman dan menjadi mode default serta yang direkomendasikan untuk penggunaan produksi.

Aspek	Privileged Container	Unprivileged Container
Pemetaan UID root	Root container = root host	Root container dipetakan ke UID non-root host
Tingkat Keamanan	Lebih rendah, risiko container escape lebih tinggi	Lebih tinggi, isolasi lebih kuat
Kompatibilitas	Lebih kompatibel untuk kasus khusus (mis. akses device tertentu)	Direkomendasikan untuk kebutuhan umum/produksi
Status Default Proxmox VE	Opsional, harus dipilih manual	Default saat pembuatan container baru

2.5 Template Container

Template container merupakan arsip terkompresi (umumnya berformat .tar.gz atau .tar.zst) yang berisi root filesystem minimal suatu distribusi Linux. Proxmox VE menyediakan daftar template resmi yang dapat diunduh langsung dari repositori bawaan melalui menu CT Templates pada storage tipe local. Setelah diunduh, template tersimpan secara lokal dan dapat digunakan berulang kali untuk membuat banyak container tanpa perlu mengunduh ulang.

Pemilihan template menentukan sistem operasi dasar container, seperti Debian 12, Ubuntu 22.04, atau Alpine Linux. Untuk kebutuhan praktikum yang selaras dengan praktik VM sebelumnya, template Debian menjadi pilihan yang konsisten karena memudahkan perbandingan langsung antara pengelolaan VM dan container pada distribusi yang sama.

2.6 Manajemen Sumber Daya Container

Salah satu keunggulan LXC adalah kemudahan mengubah alokasi sumber daya container secara dinamis. Melalui tab Resources pada antarmuka web Proxmox VE, administrator dapat mengubah jumlah core CPU maupun kapasitas memori (RAM) suatu container tanpa perlu proses instalasi ulang, bahkan pada banyak kasus dapat dilakukan tanpa mematikan container terlebih dahulu (hot-plug), tergantung dukungan dari sistem operasi di dalam container tersebut.

2.7 Snapshot dan Cloning Container

Snapshot adalah mekanisme penyimpanan kondisi (state) container pada suatu titik waktu tertentu, mencakup isi filesystem dan konfigurasinya. Snapshot memungkinkan administrator mengembalikan (rollback) container ke kondisi sebelumnya apabila terjadi kesalahan konfigurasi atau kerusakan data setelah suatu perubahan dilakukan. Fitur ini sangat berguna sebelum melakukan pembaruan (update) sistem atau instalasi paket baru yang berisiko.

Cloning adalah proses menduplikasi container menjadi container baru yang independen. Proxmox VE mendukung dua mode cloning, yaitu Full Clone (menyalin seluruh data secara utuh sehingga container hasil clone sepenuhnya independen dari sumbernya) dan Linked Clone (container hasil clone berbagi data dasar dengan sumbernya melalui referensi, sehingga lebih hemat ruang penyimpanan namun bergantung pada keberadaan container asal).

2.8 Keamanan Container

Karena LXC berbagi kernel dengan host, aspek keamanan menjadi perhatian penting. Penggunaan mode unprivileged, pembatasan kemampuan (capabilities) melalui AppArmor/SELinux profile bawaan Proxmox, serta pembatasan device yang dapat diakses container merupakan praktik terbaik untuk meminimalkan risiko container escape, yaitu kondisi di mana proses di dalam container berhasil mengakses sumber daya host di luar batas isolasinya.

BAB III

ALAT DAN BAHAN

Praktikum ini menggunakan skema virtualisasi bertingkat (nested virtualization), yaitu Proxmox VE dijalankan sebagai mesin virtual di dalam Oracle VirtualBox pada laptop, mengikuti konfigurasi yang sama seperti pada praktikum instalasi Proxmox dan pengelolaan VM sebelumnya.

No	Nama	Keterangan
1	Laptop/PC Host	Prosesor mendukung virtualisasi hardware (Intel VT-x/AMD-V aktif di BIOS)
2	Oracle VirtualBox	Aplikasi hypervisor tipe 2 sebagai wadah instalasi Proxmox VE
3	Proxmox VE	Platform virtualisasi utama, dijalankan sebagai VM di VirtualBox
4	Template Container	Debian 12 Standard (debian-12-standard), diunduh dari repositori resmi Proxmox
5	Koneksi Internet	Diperlukan untuk mengunduh template container dan paket perangkat lunak
6	Browser Web	Untuk mengakses antarmuka Proxmox VE Web Management Interface

Rencana Alokasi Sumber Daya Container

Parameter	Nilai Awal	Nilai Setelah Penyesuaian
CT ID	200	-
Hostname	debian-ct	-
Template	debian-12-standard	-
Disk (rootfs)	8 GB (local-lvm)	-
CPU Core	1	2
Memory (RAM)	512 MB	1024 MB
Swap	512 MB	512 MB
Network	eth0 – bridge vmbr0 – DHCP	-

BAB IV

LANGKAH-LANGKAH PRAKTIKUM

Bab ini menguraikan prosedur kerja standar pembuatan dan pengelolaan LXC Container pada Proxmox VE. Sebagaimana disampaikan pada bagian Batasan Laporan (1.5), langkah-langkah berikut disusun sebagai panduan pelaksanaan praktik dan belum disertai tangkapan layar hasil eksekusi aktual.

4.1 Persiapan Template Container

Sebelum sebuah container dapat dibuat, Proxmox VE memerlukan template sistem operasi yang harus diunduh terlebih dahulu. Langkah kerjanya adalah sebagai berikut:

1. Login ke Proxmox VE Web Management Interface melalui browser pada alamat <https://192.168.100.100:8006>.
2. Pada sidebar kiri, pilih storage local (proxmox), kemudian buka tab CT Templates.
3. Klik tombol Templates untuk menampilkan daftar template yang tersedia dari repositori resmi Proxmox.
4. Pilih template debian-12-standard (atau versi terbaru yang tersedia), lalu klik Download.
5. Proses pengunduhan dapat dipantau melalui panel Task History di bagian bawah antarmuka, hingga statusnya menunjukkan OK.

Template yang telah berhasil diunduh akan tersimpan secara lokal pada storage dan dapat digunakan berulang kali untuk membuat banyak container tanpa perlu mengunduh ulang.

4.2 Pembuatan Container LXC Baru

Setelah template tersedia, container baru dapat dibuat melalui langkah berikut:

1. Klik tombol Create CT di pojok kanan atas dashboard Proxmox VE.
2. Pada tab General, isi CT ID (misalnya 200), Hostname (misalnya debian-ct), dan Password root container.
3. Pada tab Template, pilih template debian-12-standard yang telah diunduh sebelumnya.
4. Pada tab Disks, tentukan ukuran rootfs (misalnya 8 GB) dan storage tujuan (local-lvm).
5. Pada tab CPU, tentukan jumlah core (misalnya 1 core sebagai nilai awal).
6. Pada tab Memory, tentukan alokasi RAM (misalnya 512 MB) dan swap (512 MB).
7. Pada tab Network, tentukan nama interface (eth0), bridge tujuan (vmbr0), dan mode alamat IPv4 (DHCP).
8. Pada tab Confirm, periksa kembali ringkasan seluruh konfigurasi sebelum menyelesaikan proses.
9. Centang opsi Start after created apabila container ingin langsung dijalankan setelah dibuat, kemudian klik Finish.

Parameter	Nilai yang Direncanakan
CT ID	200
Hostname	debian-ct
Template	debian-12-standard_12.x-1_amd64.tar.zst
Storage rootfs	local-lvm, 8 GB
CPU Core	1
Memory / Swap	512 MB / 512 MB
Network	eth0, bridge vmbr0, DHCP

4.3 Menjalankan dan Mengakses Container

Setelah container berhasil dibuat, langkah untuk menjalankan dan mengaksesnya adalah:

1. Pilih container pada sidebar kiri, kemudian buka tab Summary untuk memastikan status berubah menjadi running.
2. Buka menu Console untuk masuk ke dalam terminal container.
3. Login menggunakan akun root dan kata sandi yang telah ditentukan sebelumnya.
4. Jalankan perintah `ip a` untuk memeriksa alamat IP yang diperoleh container melalui DHCP dari bridge `vmbr0`.

4.4 Pengelolaan Siklus Hidup Container

Container LXC memiliki siklus hidup yang serupa dengan VM, namun umumnya jauh lebih responsif karena tidak melalui proses booting kernel penuh. Langkah pengelolaannya meliputi:

1. Shutdown: mematikan container secara graceful melalui tombol Shutdown pada antarmuka web. Karena container berbagi kernel dengan host, proses ini umumnya berlangsung sangat cepat dibandingkan Shutdown pada VM.
2. Start: menjalankan kembali container yang berstatus stopped melalui tombol Start.
3. Reboot: me-restart container tanpa keluar dari status running melalui tombol Reboot.
4. Seluruh riwayat operasi start/stop/reboot dapat dipantau dan diverifikasi melalui panel Task History untuk memastikan setiap perintah berstatus OK.

4.5 Instalasi Paket di Dalam Container

Salah satu keunggulan container adalah kemampuannya menjalankan manajer paket distribusi asalnya secara langsung. Langkah instalasi paket sederhana di dalam container Debian adalah sebagai berikut:

1. Perbarui daftar paket dengan perintah: `apt update`
2. Instal paket, misalnya `curl`, dengan perintah: `apt install curl -y`
3. Verifikasi keberhasilan instalasi dengan perintah: `curl --version`

Perintah	Fungsi
<code>ip a</code>	Menampilkan konfigurasi antarmuka jaringan dan alamat IP container
<code>apt update</code>	Memperbarui indeks daftar paket dari repositori
<code>apt install <paket> -y</code>	Menginstal paket perangkat lunak tanpa konfirmasi manual
<code>hostname</code>	Menampilkan nama host container, berguna untuk verifikasi identitas hasil clone
<code>df -h</code>	Memeriksa kapasitas dan penggunaan disk rootfs container

4.6 Pengaturan Ulang Sumber Daya (Resource)

Untuk menyesuaikan kapasitas container terhadap kebutuhan beban kerja, langkah pengaturan ulang sumber daya adalah:

1. Buka tab Resources pada container yang dituju.
2. Pilih parameter Memory, ubah nilai dari 512 MB menjadi 1024 MB, kemudian klik OK.
3. Pilih parameter CPU Limit/Cores, ubah nilai dari 1 menjadi 2 core, kemudian klik OK.
4. Verifikasi perubahan diterapkan dengan memeriksa kembali tab Summary atau menjalankan perintah `free -h` dan `nproc` di dalam container.

4.7 Snapshot Container

Prosedur pengambilan dan pemulihan snapshot container adalah sebagai berikut:

1. Buka tab Snapshots pada container, klik Take Snapshot.
2. Isi nama snapshot, misalnya clean-install, beserta deskripsi singkat, lalu klik Take Snapshot.
3. Lakukan perubahan pada container, misalnya membuat berkas uji dengan perintah:
`touch /root/testfile.txt`
4. Kembali ke tab Snapshots, pilih snapshot clean-install, lalu klik Rollback untuk mengembalikan container ke kondisi semula.
5. Verifikasi keberhasilan rollback dengan memeriksa kembali direktori /root menggunakan perintah `ls`, di mana berkas `testfile.txt` seharusnya sudah tidak ada.

4.8 Cloning Container

Prosedur penggandaan (cloning) container adalah sebagai berikut:

1. Pastikan container dalam keadaan stopped sebelum melakukan Full Clone.
2. Klik kanan container pada sidebar, pilih Clone.
3. Isi CT ID baru (misalnya 201), Hostname baru (misalnya `debian-ct-clone`), dan pilih mode Full Clone.
4. Klik tombol Clone, lalu pantau progres melalui Task History hingga berstatus OK.
5. Jalankan container hasil clone, buka Console, dan verifikasi identitasnya dengan menjalankan perintah `hostname` untuk memastikan container tersebut independen dari sumbernya.

BAB V

HASIL DAN PEMBAHASAN

Karena praktik langsung belum sempat dilaksanakan pada saat laporan ini disusun, bab ini menyajikan pembahasan berbasis analisis teoretis dan proyeksi hasil yang secara umum berlaku pada prosedur LXC Container di Proxmox VE, sekaligus merujuk pada pengalaman troubleshooting jaringan yang telah dialami pada praktik VM sebelumnya (Poin 2) yang relevan untuk diantisipasi.

5.1 Ringkasan Status Pelaksanaan Tahapan

Tahapan	Status	Catatan
Unduh Template	Belum dilaksanakan	Memerlukan koneksi internet stabil dari sisi Proxmox VE
Pembuatan Container	Belum dilaksanakan	Prosedur mengikuti Sub-bab 4.2
Start/Stop/Reboot	Belum dilaksanakan	Diperkirakan lebih cepat dibanding VM karena tanpa boot kernel penuh
Instalasi Paket	Belum dilaksanakan	Bergantung pada konektivitas internet container
Pengaturan Resource	Belum dilaksanakan	Umumnya dapat diterapkan tanpa restart penuh
Snapshot & Rollback	Belum dilaksanakan	Direkomendasikan sebelum melakukan perubahan berisiko
Cloning	Belum dilaksanakan	Memerlukan container dalam kondisi stopped

5.2 Antisipasi Kendala Berdasarkan Pengalaman Praktik Sebelumnya

Pada praktik pengelolaan VM sebelumnya (Poin 2), ditemukan kendala konektivitas jaringan akibat keterbatasan Bridged Adapter pada kartu Wi-Fi host dalam skema nested virtualization. Kendala serupa berpotensi turut memengaruhi container LXC apabila menggunakan bridge vbr0 yang sama, khususnya pada tahap pengunduhan template dan instalasi paket yang memerlukan akses internet keluar.

No	Potensi Kendala	Solusi yang Diantisipasi
1	Gagal mengunduh template CT karena Proxmox VE tidak memiliki akses internet keluar	Terapkan skema NAT internal (vmbri) atau pastikan koneksi Bridged Adapter berfungsi normal sebelum mengunduh template
2	Container tidak mendapatkan IP dari DHCP pada bridge vmbri	Periksa status bridge dengan perintah ip a di shell node Proxmox, atau gunakan alamat IP statis pada konfigurasi network container
3	Rollback snapshot gagal karena container masih berjalan	Pastikan container dihentikan (stopped) jika prosedur rollback mensyaratkan status tersebut
4	Clone gagal karena container sumber belum berhenti sepenuhnya	Tunggu status container benar-benar stopped melalui Task History sebelum memulai proses Clone
5	Instalasi paket gagal karena mirror repositori tidak dapat diakses	Gunakan mirror default resmi (deb.debian.org) yang secara otomatis diarahkan ke server terdekat

5.3 Analisis Perbandingan Efisiensi terhadap Virtual Machine

Secara teoretis, container LXC yang direncanakan pada Tabel 3.2 (1 core CPU, 512 MB RAM awal) akan mampu menjalankan sistem operasi Debian dengan layanan dasar secara lebih ringan dibandingkan VM 101/102 yang dibuat pada praktik sebelumnya dengan alokasi 2 core CPU dan 2048 MB RAM. Efisiensi ini konsisten dengan teori pada Sub-bab 2.3, di mana LXC tidak perlu mengemulasi perangkat keras maupun menjalankan kernel tamu terpisah.

Estimasi waktu start container juga diperkirakan jauh lebih singkat dibandingkan VM, karena tidak melalui tahapan POST BIOS/UEFI maupun proses boot kernel penuh seperti yang dialami pada instalasi VM 101 dan VM 102 sebelumnya.

Operasi	Estimasi VM (KVM)	Estimasi LXC Container
Waktu Start hingga siap digunakan	±30–60 detik	±2–5 detik
Waktu Shutdown graceful	±10–30 detik (tergantung guest agent)	±1–3 detik
Penggunaan RAM idle	Mendekati alokasi penuh (± 300–500 MB minimum OS)	Jauh lebih rendah, hanya proses yang aktif berjalan

BAB VI

PENUTUP

6.1 Kesimpulan

Berdasarkan kajian teori dan penyusunan prosedur kerja yang telah dipaparkan, dapat disimpulkan beberapa hal sebagai berikut:

1. LXC Container merupakan solusi virtualisasi tingkat sistem operasi yang jauh lebih ringan dan efisien dibandingkan VM berbasis KVM, karena berbagi kernel dengan host tanpa memerlukan emulasi perangkat keras penuh.
2. Proxmox VE menyediakan mekanisme pembuatan container yang terstandarisasi melalui template resmi, sehingga proses deployment jauh lebih cepat dibandingkan instalasi sistem operasi penuh pada VM.
3. Pengelolaan container mencakup siklus hidup dasar (start, stop, reboot), penyesuaian sumber daya secara dinamis, serta mekanisme snapshot dan cloning yang mendukung strategi backup dan replikasi lingkungan kerja.
4. Berdasarkan pengalaman praktik VM sebelumnya, kendala konektivitas jaringan akibat keterbatasan Bridged Adapter berpotensi turut memengaruhi praktik LXC Container, sehingga perlu diantisipasi sejak awal.
5. Praktik langsung pembuatan dan pengelolaan LXC Container belum sempat dilaksanakan pada saat laporan ini disusun, sehingga laporan ini bersifat sebagai kerangka acuan yang perlu ditindaklanjuti dengan eksekusi nyata.

6.2 Saran

Beberapa saran untuk tindak lanjut dan penyempurnaan laporan ini adalah:

1. Segera melaksanakan praktik langsung sesuai prosedur pada Bab IV, dengan mendokumentasikan setiap tahapan menggunakan tangkapan layar (screenshot) sebagai bukti pelaksanaan nyata.
2. Memperbarui Bab V dengan data dan hasil aktual, termasuk kendala nyata yang mungkin ditemukan selama praktik, menggantikan analisis prediktif yang saat ini disajikan.
3. Menyiapkan skema jaringan (misalnya NAT internal vmbr1) terlebih dahulu sebelum praktik, mengacu pada pengalaman troubleshooting jaringan pada praktik VM sebelumnya, agar proses pengunduhan template dan instalasi paket tidak terkendala.
4. Melakukan uji coba tambahan seperti migrasi container antar-storage atau pengujian batas atas (limit) sumber daya, untuk memperkaya pembahasan pada revisi laporan berikutnya.

DAFTAR PUSTAKA

Proxmox Server Solutions GmbH. (2024). Proxmox VE Administration Guide. Vienna: Proxmox Server Solutions GmbH.

Linux Containers Project. (2023). LXC — Linux Containers Documentation. Diakses dari dokumentasi resmi linuxcontainers.org.

Debian Project. (2024). Debian Handbook: Containers and Virtualization. Debian Documentation Project.

Kernel.org Documentation. (2023). Control Groups (cgroups) and Namespaces Documentation. The Linux Kernel Archives.

Proxmox Server Solutions GmbH. (2024). Proxmox VE: Linux Container (LXC). Dokumentasi resmi wiki Proxmox VE.

LAMPIRAN

Lampiran A — Rencana Perintah Shell untuk Verifikasi Container

Berikut kumpulan perintah shell yang direncanakan untuk digunakan selama praktik, sebagai referensi cepat pelaksanaan:

No	Perintah	Tujuan
1	pct list	Menampilkan daftar seluruh container pada node Proxmox VE
2	pct status 200	Memeriksa status container dengan CT ID 200
3	pct enter 200	Masuk ke console container 200 langsung dari shell node
4	pct config 200	Menampilkan konfigurasi lengkap container 200
5	pct snapshot 200 clean-install	Membuat snapshot container 200 melalui command line
6	pct clone 200 201 --hostname debian-ct-clone	Menggandakan container 200 menjadi container baru ID 201

Lampiran B — Perbandingan Ringkas dengan Tiga Bagian Laporan Lain

Sebagai bagian dari rangkaian empat laporan terpisah sesuai instruksi dosen, tabel berikut merangkum posisi laporan ini terhadap tiga bagian lainnya:

Bagian	Topik	Status Penyusunan
1	Virtualisasi & Instalasi Proxmox	Selesai, lengkap dengan dokumentasi
2	Membuat dan Mengelola VM	Selesai, mendokumentasikan proses troubleshooting nyata
3	Membuat dan Mengelola LXC Container	Laporan ini — disusun berbasis teori, menunggu praktik lanjutan
4	Konfigurasi Jaringan	Selesai, lengkap dengan dokumentasi